Project Interim Report

On

Gram-Sewa —

Hyperlocal Civic Issue Tracker

RU-25-10436

Frontend Frameworks (React)

SESSION: 2025-26

Submitted By–

D Rupesh Kumar

Reg No. 02033

Master of Computer Application with Full Stack in Association With Google

Under the Guidance of

Mr. Ashutosh Banergee

SWE EXPERT

SCHOOL OF INFORMATION TECHNOLOGY

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Table of Contents

1. PROJECT TITLE

Gram-Sewa: Village Service Complaint Tracking System

2. ABSTRACT

Gram-Sewa, meaning "Village Service", is a hyperlocal civic issue tracking platform designed to empower rural citizens by providing a structured digital channel for reporting local infrastructure problems.

The platform enables villagers to report issues such as damaged roads, water supply failures, electricity outages, sanitation problems, and other local civic concerns through a simple, mobile-friendly web interface.

The system is built as a Phase 1 frontend-only prototype using React, Vite, and Tailwind CSS.

It stores user and complaint data in the browser using localStorage, making it easy to demonstrate without requiring a backend server or database.

Citizens can register, log in, file complaints with image proof, and track their submitted complaints.

Authorities can access a dedicated admin dashboard to view complaints, filter records, assign priorities, update complaint status, and add resolution remarks.

Each complaint follows a transparent lifecycle: Pending, In Review, In Progress, and Resolved.

The system also includes CSV import/export functionality for data portability, seeded demo users and complaints, and an English/Hindi language switch to improve accessibility.

Gram-Sewa addresses the communication gap between rural communities and local governing bodies such as Gram Panchayats and Block Development Offices by introducing documentation, visibility, and accountability into the complaint resolution process.

3. INTRODUCTION

In rural India, civic infrastructure complaints are commonly communicated verbally to local ward members, Gram Panchayat officials, or other local representatives.

Issues such as broken roads, contaminated water supply, irregular electricity, blocked drainage, and overflowing waste often lack a formal record.

Because there is no structured documentation trail, citizens cannot easily track whether their complaints have been acknowledged, reviewed, or resolved.

The absence of a transparent reporting mechanism leads to repeated complaints, delayed resolutions, and reduced public confidence in local governance.

Gram-Sewa attempts to solve this problem by providing a simple web-based complaint tracking platform.

Citizens can file complaints with details and optional photographic evidence, while authorities can review and manage those complaints from a centralized dashboard.

This project is developed as a Phase 1 prototype. The entire application runs on the client side.

React handles the user interface, Tailwind CSS provides responsive styling, and localStorage persists all runtime data inside the browser.

A CSV import/export module allows complaint records to be saved and restored externally, reducing the limitation of browser-only storage.

4. PROBLEM STATEMENT

Rural citizens often face difficulty in reporting and tracking civic issues due to the lack of a structured digital complaint management system.

Verbal complaint reporting does not provide confirmation, status tracking, photographic evidence, or accountability.

Authorities also lack a centralized interface to organize complaints by category, priority, village, ward, and current progress.

Therefore, there is a need for a lightweight, accessible, and mobile-friendly platform where citizens can submit civic complaints and authorities can transparently manage their resolution lifecycle.

5. OBJECTIVES

The main objectives of Gram-Sewa are:

- Enable citizens to report infrastructure and civic issues with title, description, category, location, and image proof.
- Provide user registration and login for citizens and authorities.
- Store all user, complaint, and session data locally in the browser using localStorage.
- Provide a citizen dashboard where users can view and track their filed complaints.
- Provide an authority/admin dashboard to view, filter, prioritize, update, and resolve complaints.
- Implement a complete complaint lifecycle: Pending -> In Review -> In Progress -> Resolved.
- Implement CSV export to download complaint records as a portable file.
- Implement CSV import to restore complaint data from a previously exported file.
- Provide seeded demo users and sample complaints for easy project demonstration.
- Add English and Hindi language switching for improved accessibility.
- Build a responsive, mobile-first interface suitable for smartphone users.

6. SCOPE OF THE PROJECT

This Phase 1 version of Gram-Sewa is a frontend-only prototype with full UI functionality.

The application supports five complaint categories:

- Road
- Water Supply
- Electricity
- Sanitation
- Other

Users can register and log in, file complaints with optional image attachments, track complaint status, and use the dashboard based on their role.

Authority users can update complaint status, assign priority, add remarks, and filter complaint records.

Data persistence is handled through localStorage.

This means data is saved in the user's browser and remains available until the browser storage is cleared.

To improve portability, CSV export/import functionality is provided. The app also includes seeded users and complaints from JSON files for demonstration.

Future phases may introduce Express.js, MongoDB, JWT authentication, maps, notifications, and real multi-user deployment.

7. TECHNOLOGY STACK

Frontend:

- React: Used to build reusable UI components and manage application state.
- Vite: Used as the development server and build tool for fast frontend development.
- Tailwind CSS: Used for responsive, utility-first styling.
- Lucide React: Used for clean and consistent UI icons.

Storage and Data:

- localStorage: Used to store users, sessions, complaints, and language preference in the browser.
- JSON seed files: Used to provide initial demo users and sample complaints.
- CSV: Used for complaint export and import.

Development Tools:

- Node.js and npm: Used to install dependencies and run scripts.
- Visual Studio Code: Recommended IDE for development.
- Git and GitHub: Used for version control and project hosting.

Build Commands:

- npm install
- npm run dev
- npm run build

8. SYSTEM REQUIREMENTS

Hardware Requirements:

- Computer or smartphone with minimum 2 GB RAM.
- Any modern processor such as Intel, AMD, or ARM.
- Minimal storage because the app is browser-based.
- Internet connection for first-time hosted page loading.

Software Requirements:

- Operating System: Windows, Linux, macOS, Android, or iOS.
- Browser: Google Chrome 90+, Mozilla Firefox 90+, Microsoft Edge 90+, or Safari 15+.
- Development: Node.js v18+ with npm.
- Editor: Visual Studio Code.
- Version Control: Git and GitHub.

9. USER ROLES

Citizen:

- Can register and log in.
- Can submit complaints.
- Can upload image proof.
- Can view only their own complaints.
- Can track complaint status and authority remarks.

Authority/Admin:

- Can log in using seeded authority credentials.
- Can view all complaints.
- Can filter complaints by status, category, and village.
- Can assign priority.
- Can update status.
- Can add resolution remarks.
- Can export/import complaint records using CSV.

10. DEMO USERS

Admin/Authority:

Phone: 9876543210

Password: admin123

Citizen 1:

Phone: 9123456780

Password: citizen123

Citizen 2:

Phone: 7982456130

Password: citizen123

Citizen 3:

Phone: 8630914725

Password: citizen123

Note: Seed passwords are stored in the JSON seed file for demo purposes, but they are hashed before being saved into localStorage.

11. COMPLAINT STATUS DEFINITIONS

Pending:

The complaint has been submitted by the citizen but has not yet been reviewed by the authority.

In Review:

The authority has seen the complaint and is checking the details, location, and required action.

In Progress:

Work has started on the issue, such as inspection, team assignment, repair planning, or field action.

Resolved:

The issue has been completed or closed with authority remarks.

12. PRIORITY DEFINITIONS

Low:

Minor issue with limited urgency.

Medium:

Moderate issue requiring attention but not immediately critical.

High:

Important issue affecting public convenience or safety.

Critical:

Urgent issue requiring immediate action because it may affect public health, safety, or essential services.

13. METHODOLOGY

Gram-Sewa follows an iterative development methodology. Since the project is frontend-only, all data operations are performed through JavaScript and browser localStorage.

Step 1: User Registration and Login

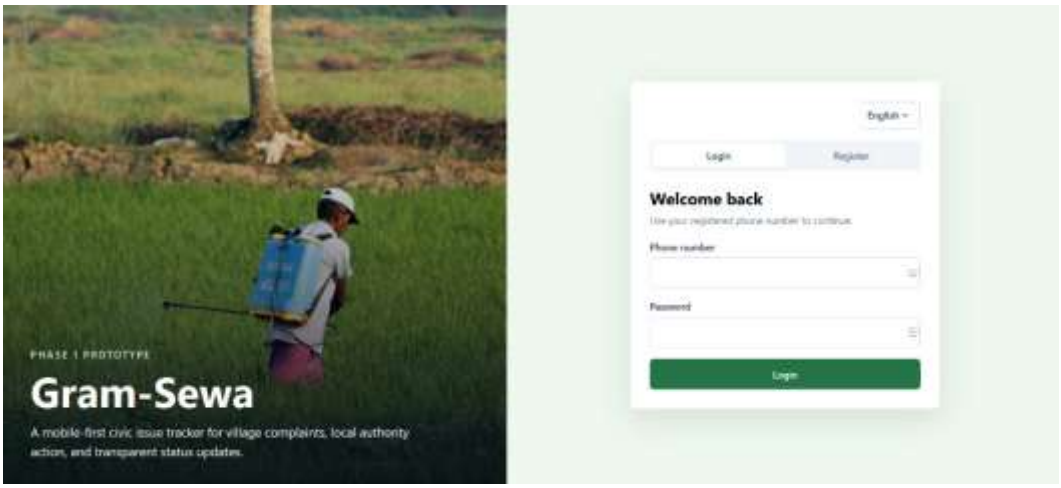

Figure: Login

Users register by entering name, phone number, password, and role.

The application hashes passwords using a lightweight SHA-256 hashing function before storing them in localStorage under the key gramsewa_users.

On login, credentials are validated against the stored user records. The current session is stored under gramsewa_current_user.

Step 2: Complaint Filing

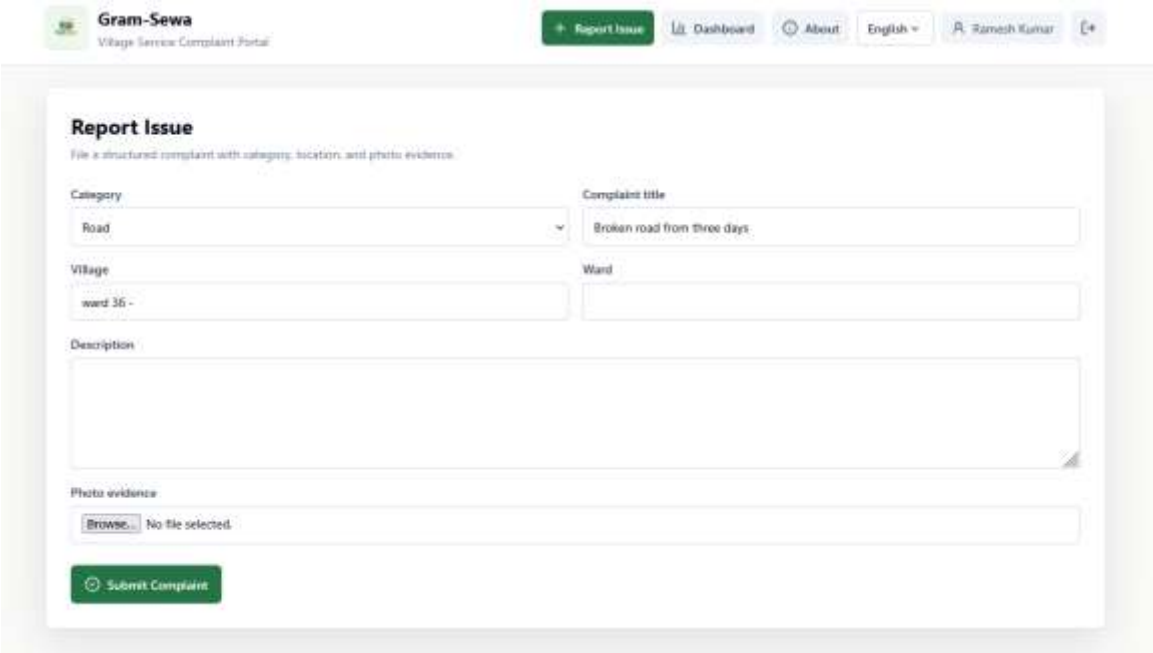

Figure: Report Issue

Citizens open the Report Issue form and enter complaint category, title, description, village, ward, and optional image proof.

Uploaded images are converted to Base64 using the FileReader API and stored as part of the complaint object.

Step 3: Complaint Storage

Each complaint receives a unique ID, default status of Pending, default priority of Medium, timestamps, and the filing user's details.

Complaints are stored in localStorage under gramsewa_complaints.

Step 4: Citizen Dashboard

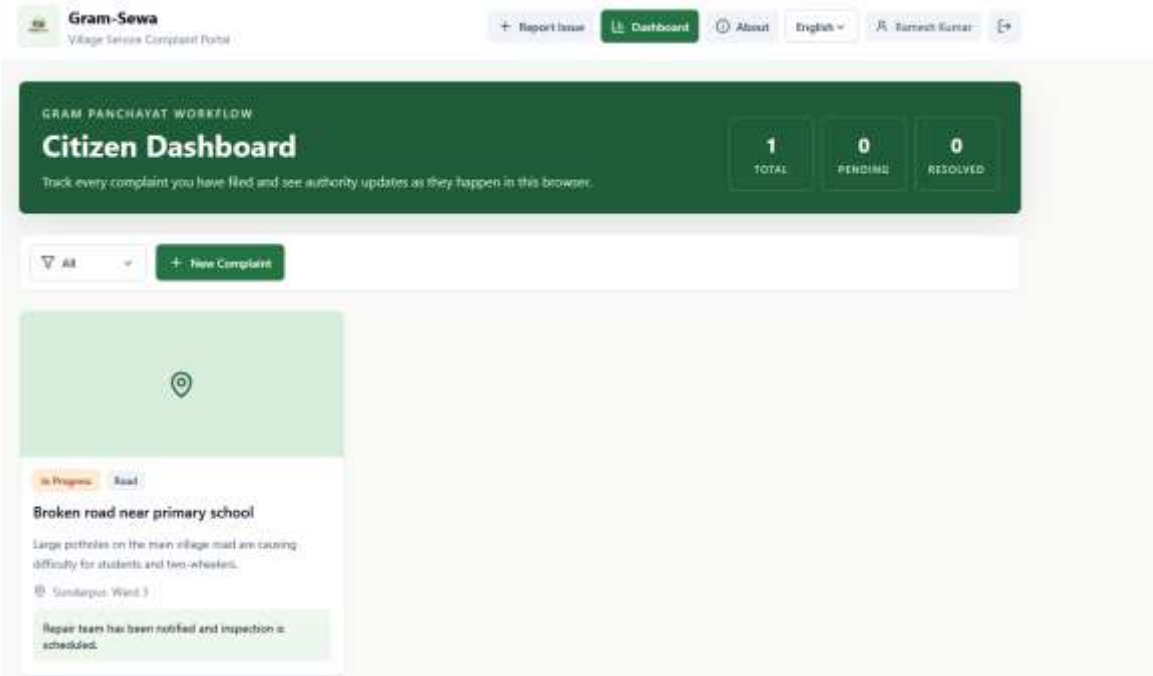

Figure: Citizen Dash

The citizen dashboard reads complaints from localStorage and filters them by the logged-in user's ID.

Each complaint appears as a card with title, category, status badge, location, image preview, and authority remarks.

Step 5: Authority Dashboard

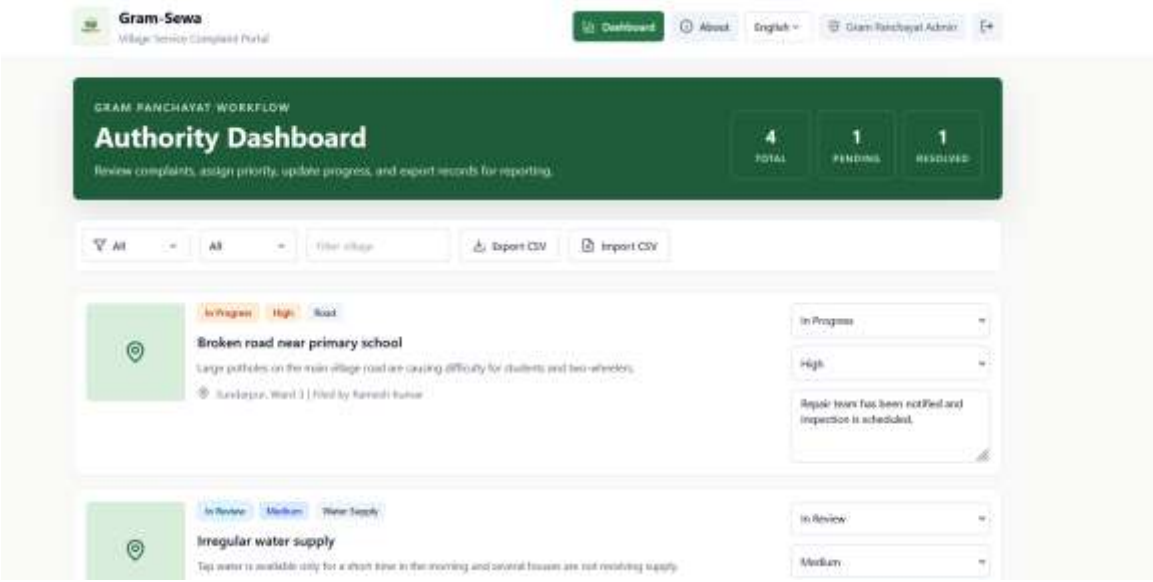

Figure: Admin Dash

The authority dashboard loads all complaints from localStorage. Authorities can filter by category, status, and village.

They can update the complaint status, assign priority, and add remarks. All changes are immediately saved back to localStorage.

Step 6: CSV Import and Export

The export function converts complaint records into CSV format and downloads the file.

The import function reads a selected CSV file, parses rows, converts them into complaint objects, and saves them into localStorage.

Step 7: Language Switching

The app supports English and Hindi. The selected language is stored in localStorage under gramsewa_language so the user preference remains available after refresh.

14. CURRENT CODE DIRECTORY STRUCTURE

Project Root:

- README.md
- package.json
- package-lock.json
- index.html
- vite.config.js
- tailwind.config.js
- postcss.config.js
- .gitignore
- gram_sewa_professional_project_report_source.txt

Source Folder:

src/

App.jsx

constants.js

csv.js

i18n.js

main.jsx

seedData.js

storage.js

styles.css

src/Assets/

logo.png

src/components/

AboutPage.jsx

Figure: About

AdminComplaintRow.jsx

AdminDashboard.jsx

AuthShell.jsx

CitizenDashboard.jsx

ComplaintCard.jsx

ComplaintForm.jsx

DashboardHero.jsx

Header.jsx

LanguageSwitch.jsx

StatusBadge.jsx

ui.jsx

src/data/

seedComplaints.json

seedUsers.json

documents/

gram_sewa_synopsis.docx

gram_sewa_synopsis.pdf

gram_sewa_synopsis.txt

15. COMPONENT AND FILE DESCRIPTION

App.jsx:

The root component of the application.

It manages the current user session, current route, complaint list state, language state, logout, login handling, and dashboard routing.

It decides whether to show the authentication screen, citizen dashboard, authority dashboard, report form, or about page.

main.jsx:

The React entry point. It mounts the App component into the root div in index.html and imports the global stylesheet.

Header.jsx:

Displays the app logo, title, navigation buttons, language switch, logged-in user information, and logout button.

It allows navigation between Dashboard, Report Issue, and About.

AuthShell.jsx:

Displays the login and registration interface.

It includes the landing text, language switch, login form, and registration form.

It calls storage functions to authenticate or register users.

CitizenDashboard.jsx:

Displays complaints filed by the logged-in citizen.

It shows summary statistics, status filtering, and complaint cards.

AdminDashboard.jsx:

Displays all complaint records for authority users.

It includes summary statistics, filters, CSV export/import controls, and complaint management rows.

ComplaintForm.jsx:

Allows citizens to submit a new complaint with category, title, description, village, ward, and image proof.

It converts uploaded images to Base64 using FileReader and saves complaint data to localStorage.

ComplaintCard.jsx:

Renders complaint information in card format for citizen dashboard display.

It shows status, category, title, description, location, image preview, and remarks.

AdminComplaintRow.jsx:

Displays each complaint in the authority dashboard.

It provides controls for changing status, assigning priority, and adding remarks.

DashboardHero.jsx:

Reusable section used at the top of dashboards to display heading, description, and key statistics.

StatusBadge.jsx:

Displays color-coded complaint status labels for Pending, In Review, In Progress, and Resolved.

LanguageSwitch.jsx:

Provides a dropdown for switching between English and Hindi.

AboutPage.jsx:

Figure: About

Explains the purpose of Gram-Sewa, key features, and meaning of complaint statuses in English or Hindi based on the selected language.

ui.jsx:

Contains reusable small UI components such as TextInput, FormTitle, and ErrorText.

constants.js:

Stores reusable arrays and style mappings for categories, statuses, priorities, and badge colors.

storage.js:

Handles all localStorage operations.

It stores users, complaints, sessions, password hashes, login, registration, and complaint updates.

seedData.js:

Loads demo users and sample complaints from JSON files. It hashes seeded passwords before saving them into localStorage and prevents duplicate seeded users by matching stable user IDs.

seedUsers.json:

Stores initial demo authority and citizen accounts.

seedComplaints.json:

Stores sample complaint data for dashboard demonstration.

csv.js:

Handles CSV export and import logic.

It converts complaint objects to CSV rows and parses CSV files back into complaint objects.

i18n.js:

Stores English and Hindi translation strings used by the application.

styles.css:

Contains Tailwind CSS directives and reusable component classes such as buttons, input fields, toolbar, and icon buttons.

16. DATA STORAGE DESIGN

localStorage Keys:

- gramsewa_users
- gramsewa_current_user
- gramsewa_complaints
- gramsewa_language

User Object:

- id
- name
- phone
- role
- passwordHash
- createdAt

Session Object:

- id
- name
- phone
- role
- createdAt

Complaint Object:

- id
- title
- description
- category
- status
- priority
- village
- ward
- filedBy
- filedByName
- remarks
- imageBase64
- createdAt
- updatedAt

17. CSV MODULE

The CSV module provides data portability for the frontend-only application.

Export reads all complaint records from localStorage and converts them into CSV with columns such as ID, Title, Description, Category, Status, Priority, Village, Ward, Filed By, Date, and Remarks.

Import reads a CSV file selected by the user, parses each row, maps values into complaint objects, and saves them into localStorage.

The application asks whether the imported data should be merged with or replace existing data.

This feature is important because localStorage is browser-specific. CSV export/import allows the project demo data to be backed up and transferred between browsers or devices.

18. AUTHENTICATION IMPLEMENTATION

Authentication is implemented for demonstration purposes using localStorage.

During registration, the user enters name, phone number, password, and role.

The password is hashed using SHA-256 through the Web Crypto API and saved as passwordHash.

During login, the entered password is hashed again and compared with stored records.

When login succeeds, the app stores a sanitized user object in gramsewa_current_user.

The password hash is not included in the active session object. Logout clears the current session from localStorage.

This approach is suitable for a frontend prototype but should be replaced with secure backend authentication, bcrypt password hashing, and JWT/session management in a production version.

19. USER INTERFACE DESIGN

The UI is designed to be simple, responsive, and mobile-first.

Tailwind CSS is used to create consistent spacing, color, typography, and layouts.

The interface uses card-based complaint views, compact filters, clear status badges, and icon-supported navigation.

The rural/civic theme is reflected through green and earth-toned colors.

The design is intentionally practical rather than overly decorative, because the target users need fast access to complaint filing and tracking workflows.

20. SAMPLE INPUT AND OUTPUT

Sample Citizen Complaint Input:

Category: Road

Title: Broken road near primary school

Description: Large potholes on the main village road are causing difficulty for students and two-wheelers.

Village: Sundarpur

Ward: Ward 3

Image: Uploaded photo

System Output:

A new complaint is created with a unique ID, status Pending, default priority Medium, current timestamp, and citizen details.

The complaint appears on the citizen dashboard.

Sample Authority Action:

The authority opens the admin dashboard, filters complaints by status, changes priority to High, updates status to In Progress, and adds remarks such as "Repair team has been notified and inspection is scheduled."

System Output:

The complaint record is updated in localStorage. The citizen dashboard displays the updated status and remarks.

21. TESTING AND VALIDATION

Testing performed:

- User registration tested for citizen and authority roles.
- Login tested with seeded demo credentials.
- Complaint submission tested with title, category, location, description, and image upload.
- Citizen dashboard tested to ensure only the logged-in user's complaints are shown.
- Authority dashboard tested to ensure all complaints are visible.
- Status updates tested from Pending to In Review, In Progress, and Resolved.
- Priority assignment tested for Low, Medium, High, and Critical.
- Remarks update tested from the admin dashboard.
- CSV export tested for complaint records.
- CSV import tested for restoring complaint data.
- Language switching tested between English and Hindi.
- Responsive layout checked for mobile and desktop widths.
- Production build verified using npm run build.

22. LIMITATIONS

- Data is stored only in the browser using localStorage.

- The app does not have a real backend database.
- Authentication is only suitable for demonstration and not production.
- Image data stored as Base64 can increase localStorage usage.
- Multi-device synchronization is not supported.
- Authority accounts are seeded locally and not managed by a secure backend.
- Notifications and map tracking are not implemented in Phase 1.

23. FUTURE ENHANCEMENTS

- Add Node.js and Express.js backend.

- Add MongoDB for centralized complaint storage.
- Replace localStorage authentication with JWT and bcrypt.
- Add map-based complaint location using Leaflet.js.
- Add SMS or email notifications for complaint updates.
- Add community upvoting for high-impact issues.
- Convert the project into a Progressive Web App.
- Add more Indian regional language support.
- Add AI-based complaint category suggestions.
- Add integration with government grievance systems.

25. CONCLUSION

Gram-Sewa successfully demonstrates a practical frontend-based civic complaint tracking system for rural governance.

The project provides a structured workflow where citizens can report local infrastructure issues with details and image proof, while authorities can manage, prioritize, and resolve complaints through a dedicated admin dashboard.

The application implements the complete complaint lifecycle from citizen reporting to authority resolution.

It uses React for modular UI development, Tailwind CSS for responsive design, localStorage for browser-based persistence, JSON seed files for demo data, and CSV import/export for data portability.

Although the current version is a Phase 1 prototype without a backend, it validates the core idea, interface design, data flow, and user experience of a rural complaint management platform.

It establishes a strong foundation for future development with Express.js, MongoDB, real authentication, notifications, maps, and production deployment.

Gram-Sewa shows how technology can improve documentation, transparency, and accountability in local governance by giving rural citizens a simple digital voice for civic grievances.

26. REFERENCES

[1] A. Banks and E. Porcello, Learning React: Modern Patterns for Developing React Apps, 2nd ed.

Sebastopol, CA: O'Reilly Media, 2022.

[2] S. Grider, Modern React with Redux, 1st ed.

San Francisco, CA: Udemy Publications, 2023.

[3] Meta Platforms, "React Documentation," React. [Online].

Available: https://react.dev

[4] Tailwind Labs, "Tailwind CSS Documentation," Tailwind CSS. [Online]. Available: https://tailwindcss.com/docs

[5] Evan You, "Vite Documentation," Vite. [Online].

Available: https://vitejs.dev/guide

[6] Mozilla Developer Network, "Web Storage API: localStorage," MDN. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[7] Mozilla Developer Network, "FileReader API," MDN. [Online].

Available: https://developer.mozilla.org/en-US/docs/Web/API/FileReader

[8] P. K. Singh and A. Sharma, "Role of ICT in Rural Governance and Development in India," International Journal of Computer Applications, vol.

120, no. 18, pp. 32-37, 2015.